

13.3 Finite-State Machines with No Output

DEFINITION 1

Suppose that A and B are subsets of V^* , where V is a vocabulary. The *concatenation* of A and B , denoted by AB , is the set of all strings of the form xy , where x is a string in A and y is a string in B .

EXAMPLE 1 Let $A = \{0, 11\}$ and $B = \{1, 10, 110\}$. Find AB and BA .

Solution: The set AB contains every concatenation of a string in A and a string in B . Hence, $AB = \{01, 010, 0110, 111, 1110, 11110\}$. The set BA contains every concatenation of a string in B and a string in A . Hence, $BA = \{10, 111, 100, 1011, 1100, 11011\}$.

Note that it is not necessarily the case that $AB = BA$ when A and B are subsets of V^* , where V is an alphabet, as Example 1 illustrates.

From the definition of the concatenation of two sets of strings, we can define A^n , for $n = 0, 1, 2, \dots$. This is done recursively by specifying that

$$A^0 = \{\lambda\},$$

$$A^{n+1} = A^n A \quad \text{for } n = 0, 1, 2, \dots$$

EXAMPLE 2 Let $A = \{1, 00\}$. Find A^n for $n = 0, 1, 2$, and 3.

Solution: We have $A^0 = \{\lambda\}$ and $A^1 = A^0 A = \{\lambda\}A = \{1, 00\}$. To find A^2 we take concatenations of pairs of elements of A . This gives $A^2 = \{11, 100, 001, 0000\}$. To find A^3 we take concatenations of elements in A^2 and A ; this gives $A^3 = \{111, 1100, 1001, 10000, 0011, 00100, 00001, 000000\}$.



$$A = \{0, 11\}, B = \{1, 10\}$$

$$BA = \{10, 111, 100, 1011\}$$

$$AB = \{01, 010, 111, 1110\}$$

$$A^n, n \rightarrow 0 \dots \infty$$

$$A^0 = \{\lambda\} \quad A^1 = A^0 A$$

$$A^1 = A^0 A = \{\lambda\}A = \{0, 11\}$$

$$A^{n+1} = A^n A$$

$$A = \{1, 00\} \quad A^n \text{ for } n = 0 \rightarrow 2$$

$$\Rightarrow A^0 = \{\lambda\}, A^1 = A^0 A = \{\lambda\}A = \{1, 00\}$$

$$A^2 = A^1 A = \{1, 00\}\{1, 00\}$$

$$= \{11, 100, 001, 0000\}$$

$$A^3 = A^2 A$$

Finite-State Automata

] We will now give a definition of a finite-state machine with no output. Such machines are also called finite-state automata, and that is the terminology we will use for them here. (Note: The singular of *automata* is *automaton*.) These machines differ from the finite-state machines studied in Section 13.2 in that they do not produce output, but they do have a set of final states. As we will see, they recognize strings that take the starting state to a final state.

DEFINITION 3

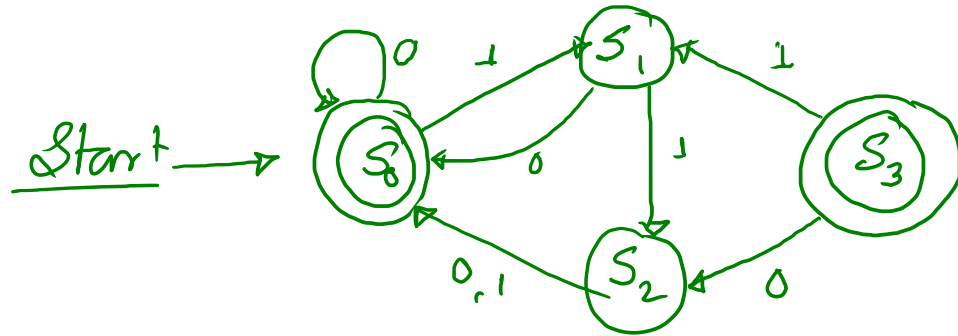
A finite-state automaton $M = (S, I, f, s_0, F)$ consists of a finite set S of *states*, a finite *input alphabet* I , a *transition function* f that assigns a next state to every pair of state and input (so that $f : S \times I \rightarrow S$), an *initial* or *start state* s_0 , and a subset F of S consisting of *final* (or *accepting states*).

We can represent finite-state automata using either state tables or state diagrams. Final states are indicated in state diagrams by using double circles.

EXAMPLE 4 Construct the state diagram for the finite-state automaton $M = (S, I, f, s_0, F)$, where $S = \{s_0, s_1, s_2, s_3\}$, $I = \{0, 1\}$, $F = \{s_0, s_3\}$, and the transition function f is given in Table 1.

TABLE 1

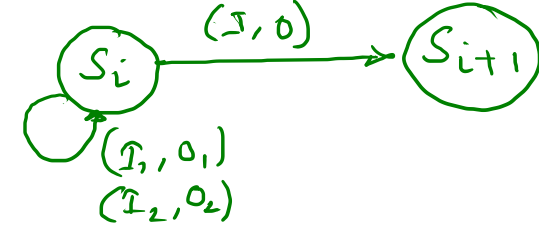
State	f	
	0	1
s_0	s_0	s_1
s_1	s_0	s_2
s_2	s_0	s_0
s_3	s_2	s_1



* FSM with output

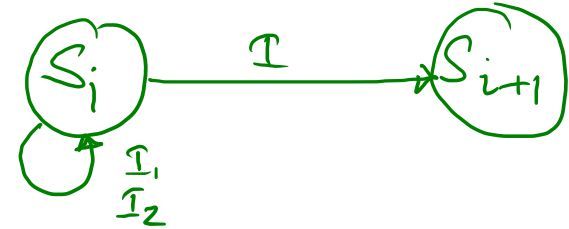
$$f(s_i, I) \rightarrow s_{i+1}$$

$$g(s_i, I) \rightarrow O$$



FSM

$$f(s_i, I) \rightarrow s_{i+1}$$

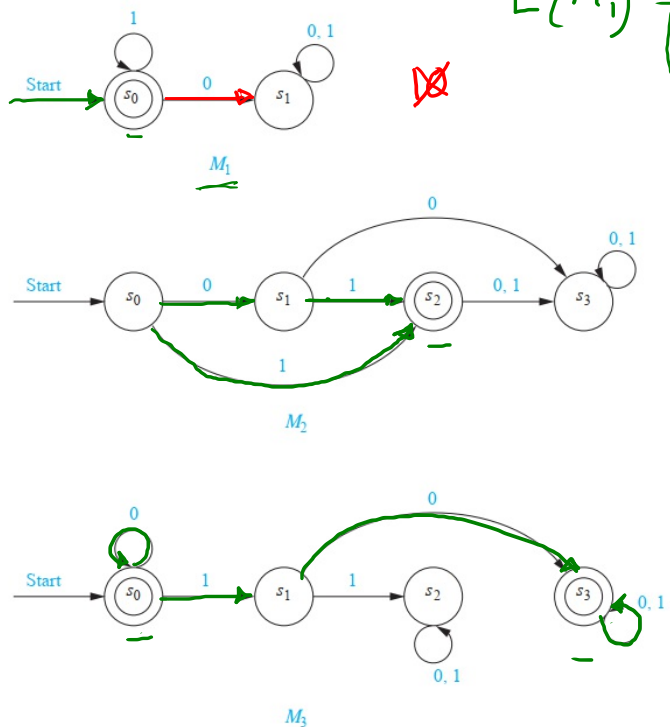


DEFINITION 4

A string x is said to be *recognized* or *accepted* by the machine $M = (S, I, f, s_0, F)$ if it takes the initial state s_0 to a final state, that is, $f(s_0, x)$ is a state in F . The *language recognized* or *accepted* by the machine M , denoted by $L(M)$, is the set of all strings that are recognized by M . Two finite-state automata are called *equivalent* if they recognize the same language.

$$L(M) \quad L(M_1) = L(M_2) \\ M_1 \equiv M_2$$

EXAMPLE 5 Determine the languages recognized by the finite-state automata M_1 , M_2 , and M_3 in Figure 2.



$$L(M_1) \rightarrow \{\lambda\}, 1, 11, 111, \dots \\ \rightarrow \{1^n \mid n = 0, 1, \dots, \infty\}$$

$$L(M_2) = \{1, 01\}$$

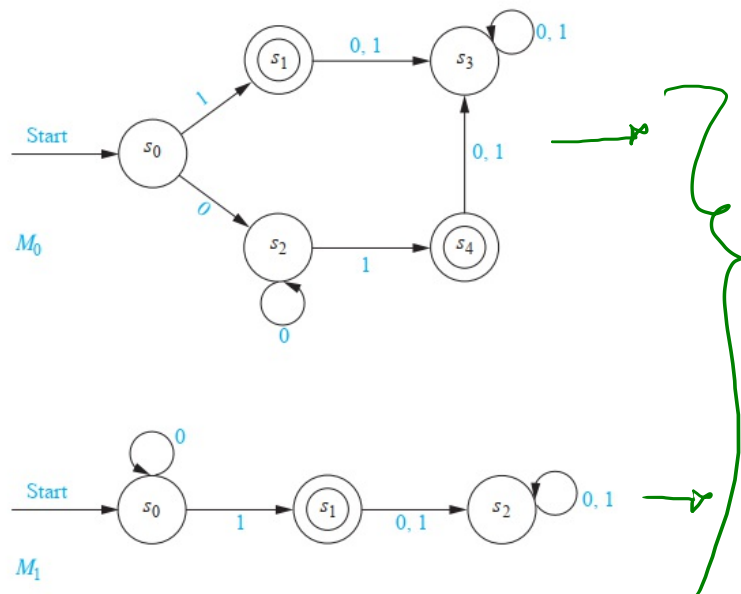
$$\begin{aligned} & \text{Handwritten diagram for } L(M_3) \text{ showing state } s_3 \text{ as a final state with a self-loop on 0.} \\ & \text{Transitions: } s_3 \xrightarrow{0} s_3 \\ & \text{Language: } L(M_3) = \{0^n, 0^n 10x \mid n = 0 \rightarrow \infty, x \text{ is any string}\} \end{aligned}$$

$$L(M_3) = \{\lambda\}, 0, 00, 000, \dots, 10, 010, 001010, \dots, 00101010, \dots, 0010101010, \dots \\ \rightarrow \{0^n, 0^n 10x \mid n = 0 \rightarrow \infty, x \text{ is any string}\}$$

FIGURE 2 Some Finite-State Automata.

EQUIVALENT FINITE-STATE AUTOMATA In Definition 4 we specified that two finite-state automata are equivalent if they recognize the same language. Example 8 provides an example of two equivalent deterministic finite-state machines.

EXAMPLE 8 Show that the two finite-state automata M_0 and M_1 shown in Figure 5 are equivalent.

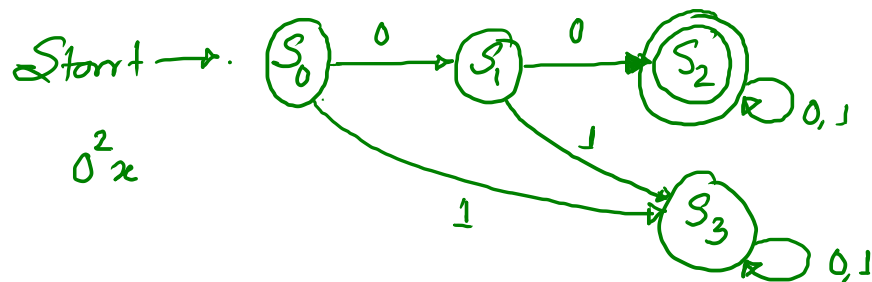


DESIGNING FINITE-STATE AUTOMATA We can often construct a finite-state automaton that recognizes a given set of strings by carefully adding states and transitions and determining which of these states should be final states. When appropriate we include states that can keep track of some of the properties of the input string, providing the finite-state automaton with limited memory. Examples 6 and 7 illustrate some of the techniques that can be used to construct finite-state automata that recognize particular types of sets of strings.

EXAMPLE 6 Construct deterministic finite-state automata that recognize each of these languages.

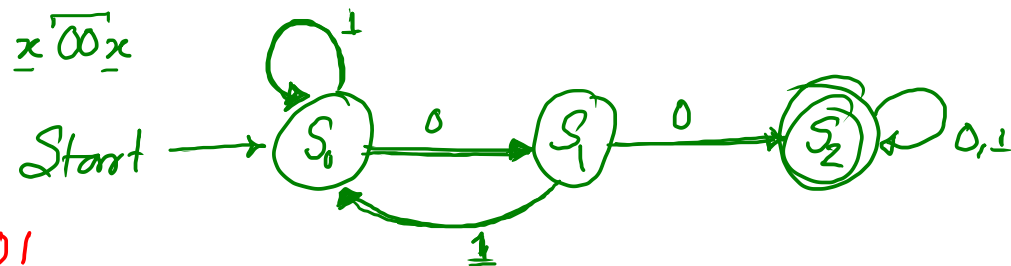
- (a) the set of bit strings that begin with two 0s
- (b) the set of bit strings that contain two consecutive 0s
- ↗ (c) the set of bit strings that do not contain two consecutive 0s
- ↗ (d) the set of bit strings that end with two 0s
- ↗ (e) the set of bit strings that contain at least two 0s

(a)



10 11 01
00, 0011, 0010

(b)



01
→ 101